

Argus Security Analysis Report

Repository: https://github.com/arnav-sutraway/Vuln_test

Total Findings: 19

1. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: admin.py, line 55

Evidence: query = f"SELECT * FROM audit_logs WHERE action LIKE '%{search_term}%'"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

2. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: auth.py, line 22

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

3. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: auth.py, line 39

Evidence: + "" WHERE email = ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

4. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: database.py, line 11

Evidence: query = "SELECT * FROM users WHERE username = " + username + ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

5. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: database.py, line 24

Evidence: query = f"SELECT * FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

6. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: database.py, line 39

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

7. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: database.py, line 74

Evidence: query = f"SELECT * FROM products ORDER BY {sort_column}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

8. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: services\reports.py, line 10

Evidence: `f"WHERE type = '{report_type}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

9. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: services\reports.py, line 24

Evidence: query = f"SELECT * FROM {table_name}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

10. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: services\users.py, line 10

Evidence: "WHERE email = " + email + " "

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

11. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 82.0%

Location: services\users.py, line 24

Evidence: query = f"DELETE FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

12. Hardcoded API Key in auth.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: auth.py, line 8

Evidence: `API_KEY = "ARGUS_FAKE_API_KEY_123456789"`

Description: The hardcoded API key in the `auth.py` file is used for authenticating to an external service. This could potentially expose sensitive information if the key is compromised.

Recommendation: Remove the hardcoded API key from source code and use environment variables or a secure secrets-management system.

13. Hardcoded Database Password in config.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 16

Evidence: DATABASE_PASSWORD = "argus_fake_postgres_password"

Description: The hardcoded database password in the `config.py` file is used to connect to a database. This could potentially expose sensitive information if the password is compromised.

Recommendation: Remove the hardcoded database password from source code and use environment variables or a secure secrets-management system.

14. Hardcoded JWT Signing Secret in auth.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: auth.py, line 48

Evidence: JWT_SECRET = "ARGUS_FAKE_JWT_SIGNING_SECRET"

Description: The hardcoded JWT signing secret in the `auth.py` file is used to sign and verify JSON Web Tokens. This could potentially expose sensitive information if the secret is compromised.

Recommendation: Remove the hardcoded JWT signing secret from source code and use environment variables or a secure secrets-management system.

15. Hardcoded Stripe Secret Key in config.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 21

Evidence: STRIPE_SECRET_KEY = "sk_test_ARGUS_FAKE_STRIPE_KEY_123456"

Description: The hardcoded Stripe secret key in the `config.py` file is used to interact with the Stripe API. This could potentially expose sensitive information if the key is compromised.

Recommendation: Remove the hardcoded Stripe secret key from source code and use environment variables or a secure secrets-management system.

16. Hardcoded AWS Access Key ID in config.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 24

Evidence: `AWS_ACCESS_KEY_ID = "AKIAARGUSFAKEACCESSKEY"`

Description: The hardcoded AWS access key ID in the `config.py` file is used to authenticate with AWS services. This could potentially expose sensitive information if the key is compromised.

Recommendation: Remove the hardcoded AWS access key ID from source code and use environment variables or a secure secrets-management system.

17. Hardcoded AWS Secret Access Key in config.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 27

Evidence: `AWS_SECRET_ACCESS_KEY = "ARGUS_FAKE_AWS_SECRET_KEY_123456789"`

Description: The hardcoded AWS secret access key in the `config.py` file is used to authenticate with AWS services. This could potentially expose sensitive information if the key is compromised.

Recommendation: Remove the hardcoded AWS secret access key from source code and use environment variables or a secure secrets-management system.

18. Hardcoded GitHub Token in config.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 34

Evidence: GITHUB_TOKEN = "ghp_ARGUS_FAKE_GITHUB_TOKEN_123456789"

Description: The hardcoded GitHub token in the `config.py` file is used to interact with the GitHub API. This could potentially expose sensitive information if the token is compromised.

Recommendation: Remove the hardcoded GitHub token from source code and use environment variables or a secure secrets-management system.

19. Hardcoded Encryption Key in config.py

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 37

Evidence: ENCRYPTION_KEY = "ARGUS_FAKE_ENCRYPTION_KEY_NOT_REAL"

Description: The hardcoded encryption key in the `config.py` file is used to encrypt and decrypt data. This could potentially expose sensitive information if the key is compromised.

Recommendation: Remove the hardcoded encryption key from source code and use environment variables or a secure secrets-management system.